Kapitel 6: Funktionen

Mathematische Grundlagen der Informatik

Prof. Dr. Andreas Vogt

Fachhochschule
Nordwestschweiz

Funktionen

Definition:

Es seien A, B Mengen. Eine Relation F von A nach B heisst Funktion von A nach B, wenn für alle $x \in A$ genau ein $y \in B$ existiert mit xFy.

Für das (eindeutig bestimmte) Element y mit xFy schreiben wir $F(x)$.

Wir schreiben $F : A \to B$ um zu zeigen, dass F eine Funktion von A nach B ist.

A heisst Definitionsbereich und B Wertebereich von F.

Beispiele:

(1) Es seien $A = \{\text{alle Studierende dieser Klasse}\}$ und $B = \{\text{alle möglichen Noten}\}$

Dann ist F mit xFy wenn y die Note von x in diesem Modul ist, eine Funktion von A nach B.

Für einen Studierenden x ist $F(x)$ also die zugehörige Note.

(2) Es sei $F = \left\{(x, y) \in \mathbb{R}^2 \mid y = 2 \cdot x\right\}$.

Dann ist F eine Funktion von $\mathbb{R}$ nach $\mathbb{R}$

Für $x \in \mathbb{R}$ ist $F(x) = 2 \cdot x$.

(3) Es sei $F = \left\{(x, y) \in \mathbb{R}^2 \mid y^2 = x\right\}$.

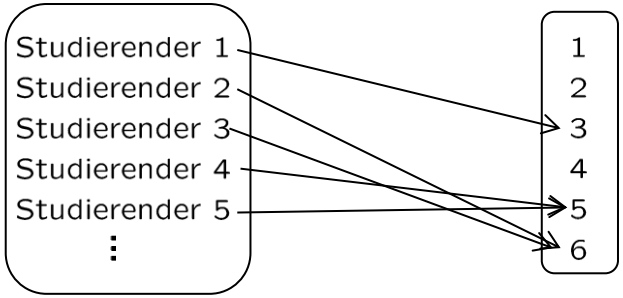
Dann ist F keine Funktion, da etwa $(1, -1) \in F$ und $(1, 1) \in F$.

Darstellung von Funktionen

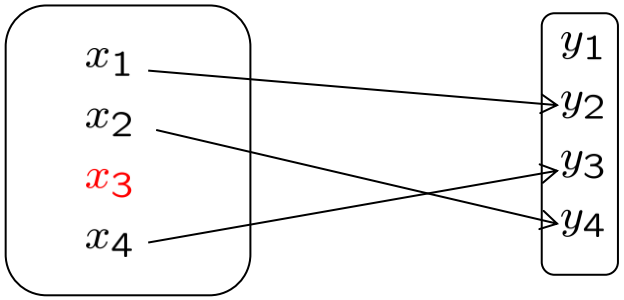
In der Pfeil-Darstellung, die wir für Relationen kennengelernt haben, erkennt man eine Funktion $F : A \to B$ daran, dass aus jedem Element von A genau ein Pfeil ausgeht.

Beispiel: $A = \{\text{alle Studierende dieser Klasse}\}$ und $B = \{\text{alle möglichen Noten}\}$

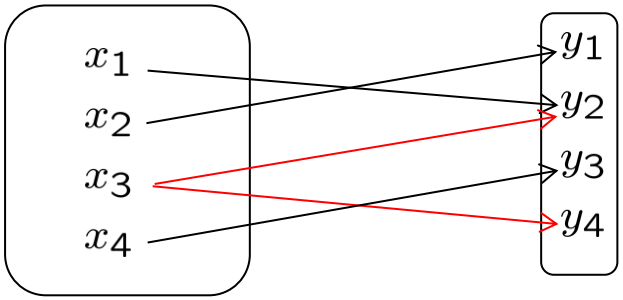
Für einen Studierenden x sei $F(x)$ die zugehörige Note.



Beispiel:



Keine Funktion

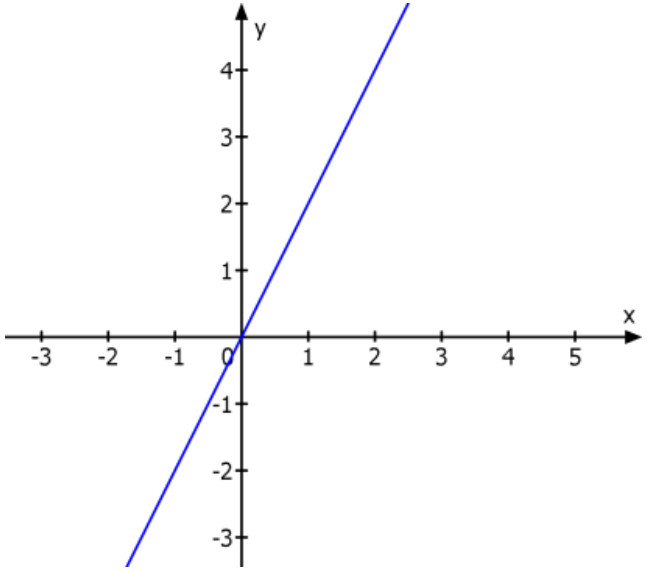


Keine Funktion

Darstellung von Funktionen

Wenn Definitions- und Wertebereich einer Funktion F aus Zahlen besteht, stellt man Funktionen oft in einem Koordinatensystem dar, wobei die horizontale Achse den Definitionsbereich und die vertikale Achse den Wertebereich enthält.

Beispiel: $F = \{(x, y) \in \mathbb{R} \times \mathbb{R} \mid y = 2 \cdot x\}$



Funktionen

Aufgabe: Entscheiden Sie, ob folgende Relationen Funktionen sind.

(1) $A = \{\text{alle Städte der Schweiz}\}$, $B = \{\text{alle Kantone der Schweiz}\}$

$F = \{(x, y) \in A \times B \mid x \text{ liegt in } y\}$

Ja, jede Stadt der Schweiz liegt in genau einem Kanton der Schweiz.

(2) $A = \{\text{alle Städte der Welt}\}$, $B = \{\text{alle Kantone der Schweiz}\}$

$F = \{(x, y) \in A \times B \mid x \text{ liegt in } y\}$

Nein, da Berlin in keinem Kanton der Schweiz liegt.

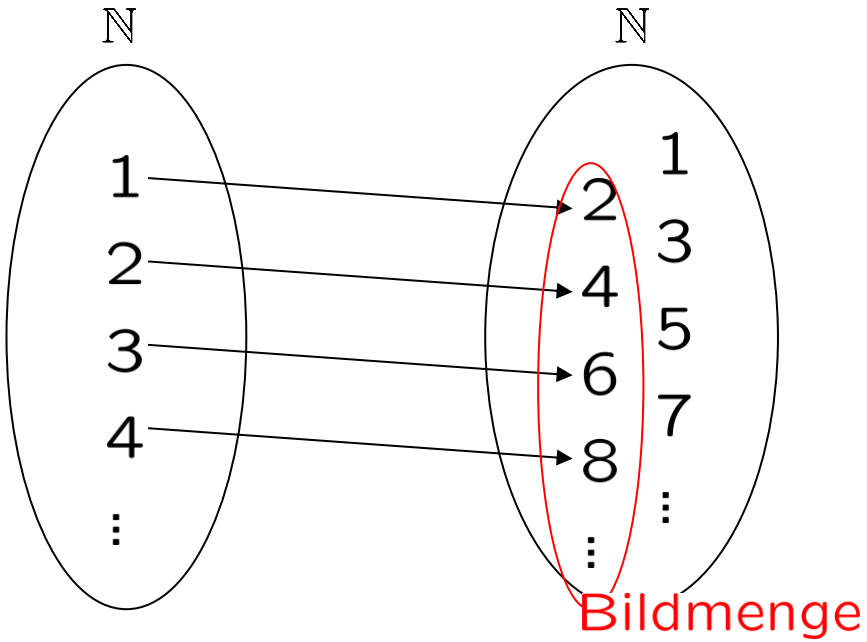
(3) $A = \{\text{alle Kantone der Schweiz}\}$, $B = \{\text{alle Städte der Schweiz}\}$

$F = \{(x, y) \in A \times B \mid x \text{ enthält } y\}$

Nein, da (Aargau, Rheinfelden)$\in F \wedge$(Aargau, Brugg)$\in F$.

Bildmenge

Die Funktion $f : \mathbb{N} \to \mathbb{N}$ sei durch $f(x) = 2x$ definiert.



In der Wertemenge existiert nur für eine gerade Zahl $y \in \mathbb{N}$ ein Urbild (Argument, Input) $x \in \mathbb{N}$, so dass $y = f(x) = 2x$ gilt. Diese Menge wird Bildmenge der Funktion f genannt.

Definition:
Für eine Funktion $f : A \to B$ heisst die Menge $f(A) = \{y \in B \mid \exists x \in A : f(x) = y\}$ Bildmenge von f.

Betragsfunktion

Definition:

Die Funktion $f : \mathbb{Z} \to \mathbb{Z}$ mit $f(x) = |x| = \begin{cases} x, & \text{für } x \geq 0, \\ -x, & \text{für } x < 0. \end{cases}$ heisst Betragsfunktion.

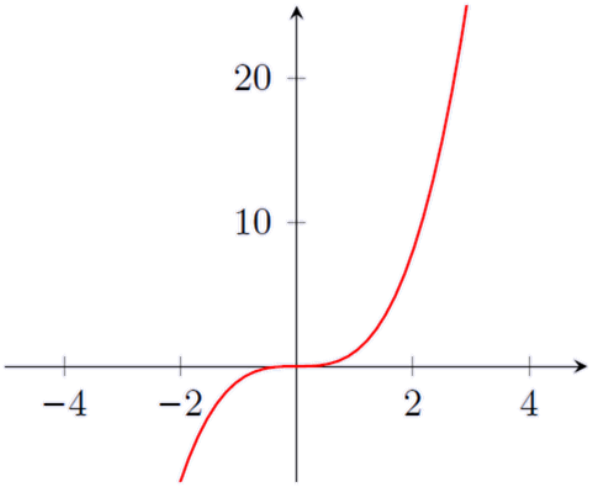
Es ist etwa $|7| = 7$, $|-8| = 8$, $|0| = 0$.

Für die Bildmenge gilt $f(\mathbb{Z}) = \mathbb{N}_0$.

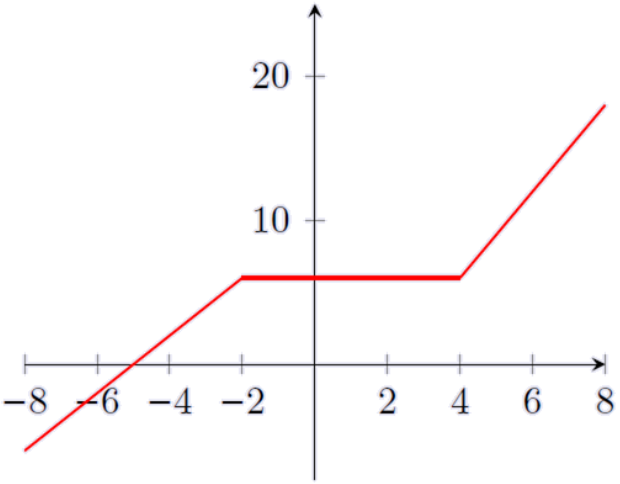
Monotonie

Definition:

Eine Funktion $f : A \to B$ mit $A, B \subseteq \mathbb{R}$ heisst streng monoton wachsend, falls für alle $x_1 < x_2$ gilt $f(x_1) < f(x_2)$.



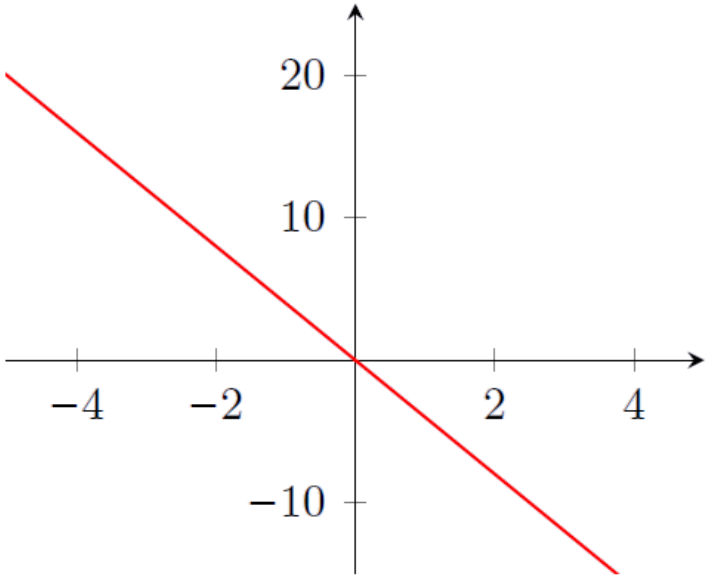
Eine Funktion $f : A \to B$ mit $A, B \subseteq \mathbb{R}$ heisst monoton wachsend, falls für alle $x_1 < x_2$ gilt $f(x_1) \leq f(x_2)$.



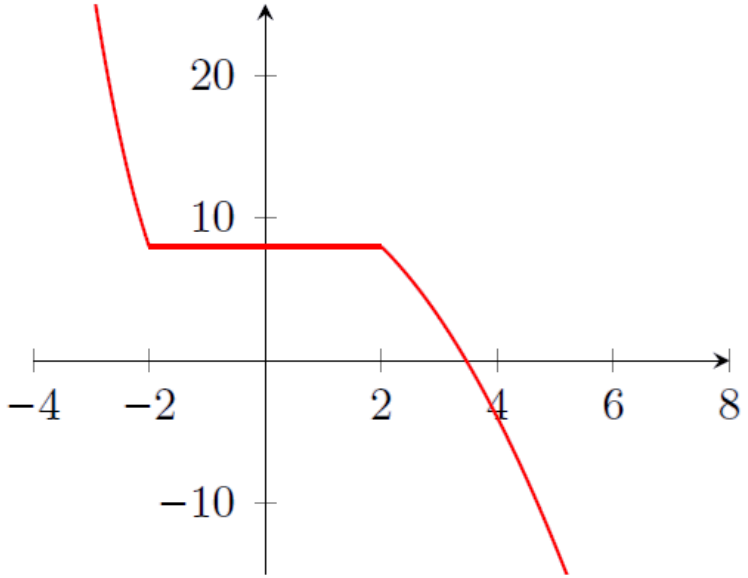
Monotonie

Definition:

Eine Funktion $f : A \to B$ mit $A, B \subseteq \mathbb{R}$ heisst streng monoton fallend, falls für alle $x_1 < x_2$ gilt $f(x_1) > f(x_2)$.



Eine Funktion $f : A \to B$ mit $A, B \subseteq \mathbb{R}$ heisst monoton fallend, falls für alle $x_1 < x_2$ gilt $f(x_1) \geq f(x_2)$.



Eigenschaften von Funktionen

Definition:

Es sei $F : A \to B$ eine Funktion von A nach B. Dann heisst F

(1) injektiv, wenn $\forall x_0, x_1 \in A : F(x_0) = F(x_1) \Rightarrow x_0 = x_1$

Oder anders ausgedrückt: $x_0 \neq x_1 \Rightarrow F(x_0) \neq F(x_1)$

(2) surjektiv, wenn $\forall y \in B \, \exists x \in A : F(x) = y$

(3) bijektiv, wenn F injektiv und surjektiv ist

Zur Erinnerung:
$p \Rightarrow q$ ist äquivalent zu $\neg q \Rightarrow \neg p$

Beispiele:

(1) Es seien $A = \{$alle Studierende dieser Klasse$\}$ und
$B = \{$alle möglichen Email-Adressen$\}$

Es sei $F : A \to B$ definiert durch $F(x) :=$ FHNW-Email-Adresse von x.

Dann ist F injektiv, da verschiedene Studierende verschiedene FHNW-Email-Adressen haben.

F ist nicht surjektiv, da es kein $x \in A$ gibt mit $F(x) =$andreas.vogt@fhnw.ch.

Da F nicht surjektiv ist, ist F auch nicht bijektiv.

Eigenschaften von Funktionen

(2) Es seien $A = \{\text{alle Städte der Schweiz}\}$ und $B = \{\text{alle Kantone der Schweiz}\}$.

Es sei $F : A \to B$ definiert durch $F(x) :=$ Kanton, in dem x liegt.

Dann ist F nicht injektiv, da etwa Rheinfelden $\neq$ Windisch,
aber $F(\text{Rheinfelden}) = F(\text{Windisch}) = \text{Aargau}$.

F ist surjektiv, da es für jedes $y \in B$ (also jeden Kanton) ein $x \in A$ gibt (also eine Stadt) mit $F(x) = y$ (d.h. die Stadt x liegt im Kanton y).

Da F nicht injektiv ist, ist F auch nicht bijektiv.

(3) Es sei $F = \{(x, y) \in \mathbb{R} \times \mathbb{R} \mid y = 2 \cdot x\}$.

F ist injektiv, denn: $F(x_0) = F(x_1)$ bedeutet $2 \cdot x_0 = 2 \cdot x_1$, also $x_0 = x_1$.

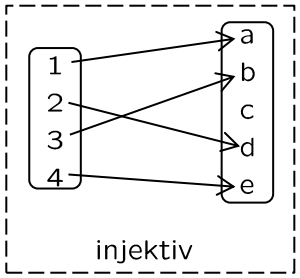
F ist surjektiv, denn zu jedem $y \in \mathbb{R}$ existiert ein $x \in \mathbb{R}$ mit $F(x) = y$, und zwar $x = \frac{1}{2} \cdot y$.

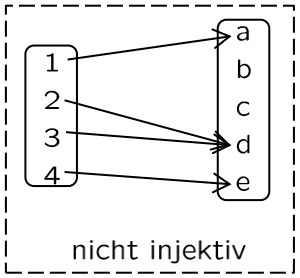
Da F injektiv und surjektiv ist, ist F bijektiv.

Eigenschaften von Funktionen

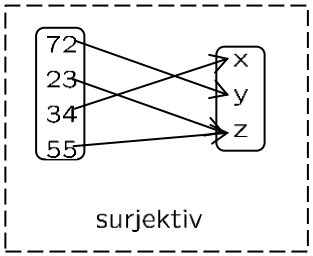
Im Pfeildiagramm bedeutet

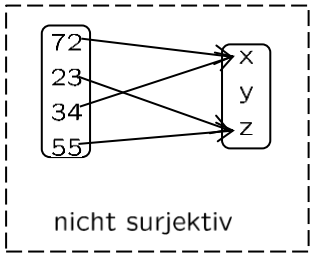
injektiv, dass bei jedem Element auf der rechten Seite höchstens ein Pfeil ankommt:



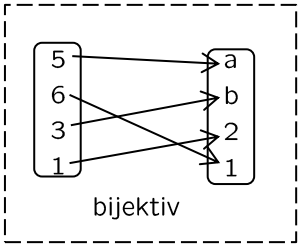


surjektiv, dass bei jedem Element auf der rechten Seite mindestens ein Pfeil ankommt:





bijektiv, dass bei jedem Element auf der rechten Seite genau ein Pfeil ankommt:



(Bei jeder Funktion entspringt aus jedem Element auf der linken Seite genau ein Pfeil!)

Verknüpfung von Funktionen

Satz:

Es sei $F : A \to B$ eine Funktion von A nach B und $G : B \to C$.

Dann ist die Verknüpfung $G \circ F$ wieder eine Funktion, und zwar von A nach C.

Für die Funktion $G \circ F$ gilt $(G \circ F)(a) = G(F(a))$.

Die Funktion, die näher am Argument steht, wird zuerst angewendet!

Begründung:

Da F eine Funktion ist, existiert zu jedem $a \in A$ genau ein $b \in B$ mit aFb.

Es ist also $b = F(a)$.

Und da G eine Funktion ist, existiert genau ein $c \in C$ mit bGc.

Es ist also $c = G(b) = \boxed{G(F(a))}$

Definitionsgemäss ist also $a(G \circ F)c$.

Es ist also $c = \boxed{(G \circ F)(a)}$

also gleich

Beispiel:

Es seien $F : \mathbb{R} \to \mathbb{R}$ definiert durch $F(x) = x^2$ und $G : \mathbb{R} \to \mathbb{R}$ definiert durch $G(x) = x + 1$.

Dann gilt $(G \circ F)(x) = G(F(x)) = G(x^2) = x^2 + 1$

und $(F \circ G)(x) = F(G(x)) = F(x + 1) = (x + 1)^2$.

Es gilt z.B. $(G \circ F)(3) = 10$ und $(F \circ G)(3) = 16$.

I.A. ist die Verknüpfung von Funktionen also nicht kommutativ!

inverse Funktionen

Wir betrachten folgende Relation:

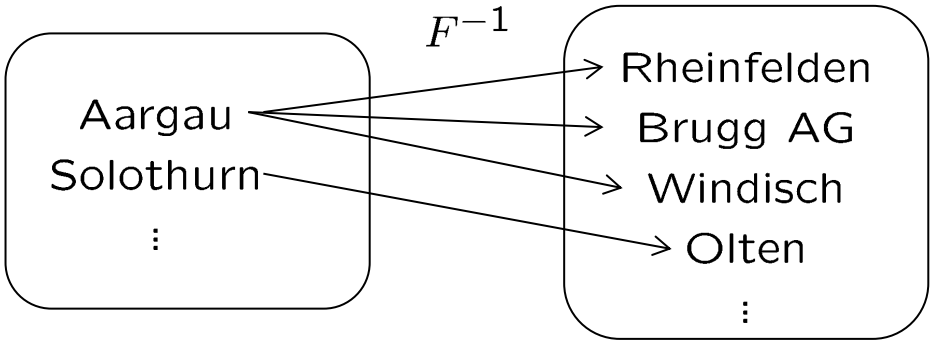
$A = \{\text{alle Städte der Schweiz}\}$, $B = \{\text{alle Kantone der Schweiz}\}$

$F = \{(x, y) \in A \times B \mid x \text{ liegt in } y\}$

Diese Relation ist sogar eine Funktion.

Ist die inverse Relation auch eine Funktion? $F^{-1} = \{(y, x) \in B \times A \mid y \text{ enthält } x\}$

Nein, jedes $y \in B$, also jeder Kanton, steht mit mehrere Städten in Relation.



Wann ist die inverse Relation einer Funktion wieder eine Funktion?

inverse Funktionen

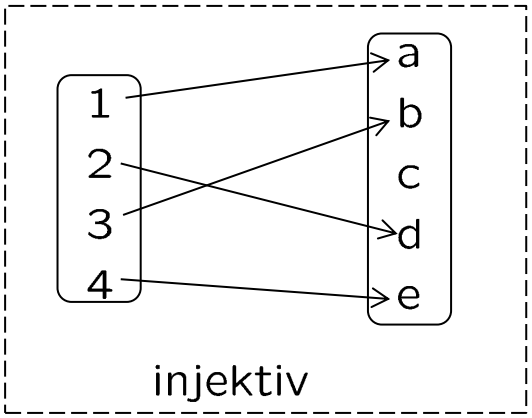
Wann ist die inverse Relation einer Funktion wieder eine Funktion?

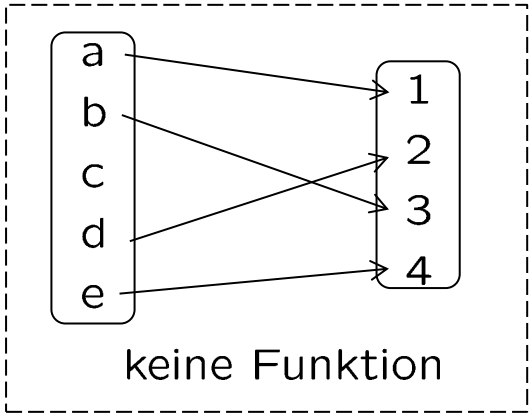
Bei einer Funktion darf aus jedem Element der Definitionsmenge nur ein Pfeil ausgehen.

Damit das bei F^{-1} der Fall ist, darf bei F jedes Element der Wertemenge höchstens einen Pfeil abbekommen.

F muss also injektiv sein!

Reicht das?





Nein, bei einer Funktion muss aus jedem Element der Definitionsmenge ein Pfeil ausgehen.

Damit dies bei F^{-1} der Fall ist, muss jedes Element der Wertemenge von F einen Pfeil abbekommen. F muss also surjektiv sein!

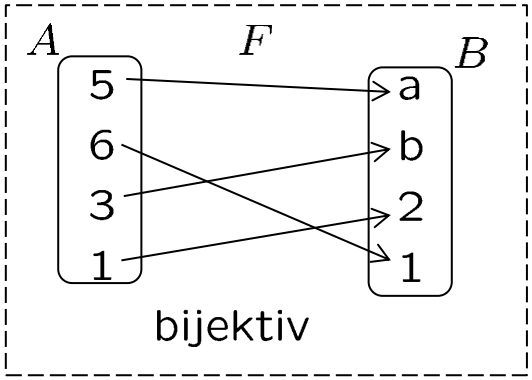
➡ Damit $F^{-1} : B \to A$ wieder eine Funktion ist, muss $F : A \to B$ bijektiv sein!

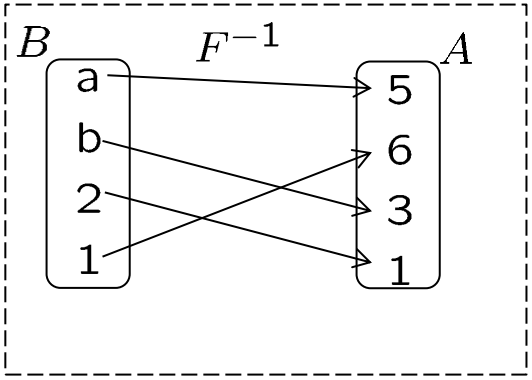
inverse Funktionen

Satz:

Es sei $F : A \to B$ eine **bijektive** Funktion. Dann ist $F^{-1} : B \to A$ eine Funktion, die sogenannte inverse Funktion zu F oder Umkehrfunktion von F.

Beispiele: (1)





(2) $F : \mathbb{R} \to \mathbb{R}$ mit $F(x) = 2 \cdot x$ ist bijektiv.

Es ist $F^{-1} : \mathbb{R} \to \mathbb{R}$ mit $F^{-1}(y) = \frac{1}{2}y$.

Die Umkehrfunktion wird bestimmt, in dem man die Gleichung $F(x) = y$ nach x auflöst.

Beispiel:

Die Funktion $f : \mathbb{R} \to \mathbb{R}$, definiert durch $f(x) = 2x + 1$ ist bijektiv. Wir berechnen die Umkehrfunktion: $f(x) = y \Leftrightarrow 2x + 1 = y \Leftrightarrow 2x = y - 1 \Leftrightarrow x = \frac{1}{2}(y - 1)$

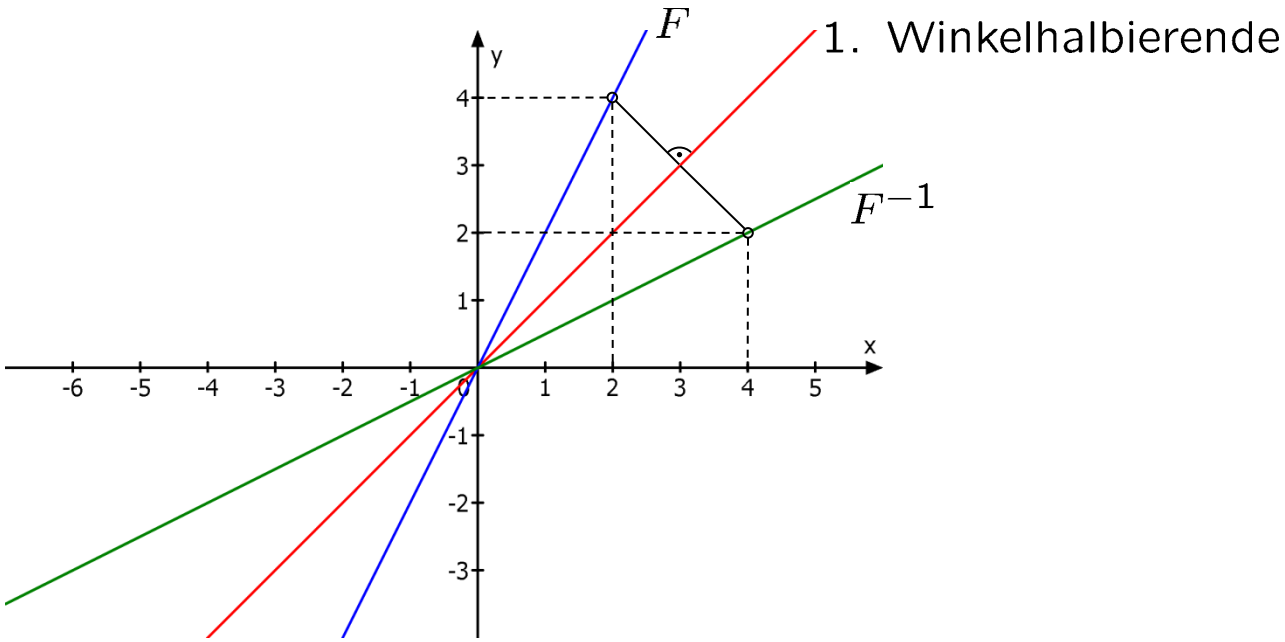
Es ist also $f^{-1}(y) = \frac{y-1}{2}$. Meist nennt man die Variable y zu x um: $f^{-1}(x) = \frac{x-1}{2}$.

inverse Funktionen

Für eine Funktion F, deren Definitionsbereich und Wertebereich aus Zahlen bestehen, ergibt sich die Darstellung von F^{-1} im Koordiantensystem aus der von F durch Spiegelung an der ersten Winkelhalbierenden.

Beispiel:

Für $F : \mathbb{R} \to \mathbb{R}$ mit $F(x) = 2 \cdot x$ ist $F^{-1} : \mathbb{R} \to \mathbb{R}$ mit $F^{-1}(y) = \frac{1}{2}y$.



Asymptotisches Verhalten von Funktionen

Die Laufzeit eines Programms hängt typischerweise von der Eingabegrösse ab.

```
int[] array = {4, 7, 2, 9, 1, 5};

int sum = 0;

for (int i = 1; i < array.length; i++)
    sum = sum + array[i];


System.out.println("Die Summe ist: " + sum);
```

Der nebenstehende Algorithmus summiert alle Elemente in einem Array auf.

Für jedes Element im Array wird eine Addition ausgeführt.

Ausserdem haben wir noch drei Schritte ausserhalb der Schleife.

Wenn wir ein Array mit n Elementen haben, dann führt der Algorithmus also $f(n) = n + 3$ Befehle aus.

Definition:

Die Laufzeit eines Programms wird als Anzahl der ausgeführten Befehle in Abhängigkeit der Eingabelänge n durch eine Funktion $f(n)$ definiert.

Asymptotisches Verhalten von Funktionen

Problem:

Die genaue Anzahl an ausgeführten Befehlen für Eingaben einer festen Länge n zu zählen ist sehr aufwendig, beziehungsweise fast nicht möglich.

Das hängt auch von der Programmiersprache ab. Im vorigen Beispiel haben wir eine for-Schleife gehabt. Hier wird auf i immer eins draufaddiert. Ausserdem wird i mit der Array-Länge verglichen. Hier haben wir also noch ein paar mehr Schritte, die wir bisher gar nicht gezählt haben.

Die genaue Anzahl Schritte interessiert uns auch gar nicht. Wir fokussieren uns auf das Wachstumsverhalten der Funktion, insbesondere für grössere Eingaben.

Verdoppelt sich die Laufzeit, wenn ich die Eingabe verdoppele? Oder verzehntfacht sich die Laufzeit bei Verdoppelung der Eingabe?

Das ist entscheidend für die Praktikabilität eines Algorithmus!

Wir teilen die Funktionen so in Klassen ein, dass sich die Funktionen in einer Klasse ähnlich verhalten.

Asymptotisches Verhalten von Funktionen

Definition: Es seien $f, g : \mathbb{N} \to \mathbb{R}$ zwei Funktionen. Dann ist

$f(n) \in \mathcal{O}(g(n))$

Es existiert ein $n_0 \in \mathbb{N}$ und ein $c \in \mathbb{N}$, so dass für alle $n \geq n_0$ gilt: $f(n) \leq c \cdot g(n)$.

f wächst asymptotisch nicht schneller als g.

$f(n) \in \Omega(g(n))$

Es existiert ein $n_0 \in \mathbb{N}$ und ein $d \in \mathbb{N}$, so dass für alle $n \geq n_0$ gilt: $f(n) \geq \frac{1}{d} \cdot g(n)$.

f wächst asymptotisch mindestens so schnell wie g.

$f(n) \in \Theta(g(n))$

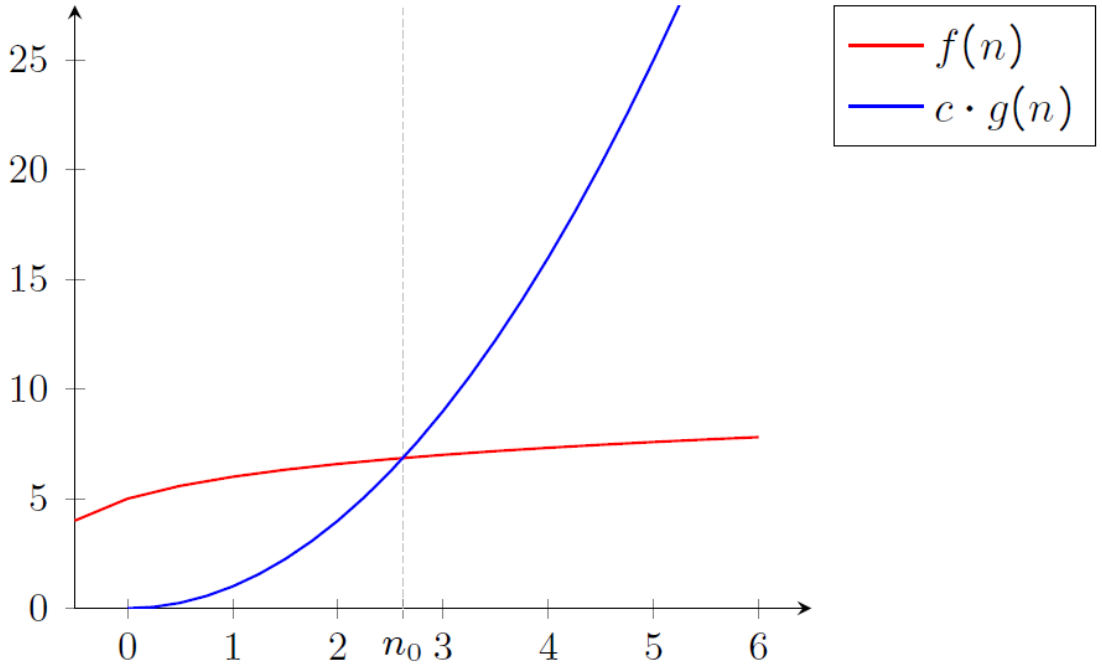
Es gilt $f(n) \in \mathcal{O}(g(n))$ und $f(n) \in \Omega(g(n))$.

f und g sind asymptotisch gleich.

Asymptotisches Verhalten von Funktionen

Wir beschränken uns im folgenden auf obere Schranken.

Es gilt $f(n) \in \mathcal{O}(g(n))$, wenn eine natürliche Zahl n_0 existiert, so dass für alle $n \geq n_0$ die Funktionswerte $f(n)$ immer kleiner oder gleich dem Funktionswert $g(n)$ multipliziert mit einer Konstanten c ist:



Beispiele:

(1) $\underbrace{7n+4}_{f(n)} \in \mathcal{O}(\underbrace{n}_{g(n)})$

Für alle $n \geq n_0 = 4$ ist $f(n) = 7n + 4 \leq 7n + n = 8n = c \cdot g(n)$.

(2) $\frac{1}{2}n^2 + 5n - 4 \in \mathcal{O}(n^2)$

Für alle $n \geq 1$ ist $\frac{1}{2}n^2 + 5n - 4 \leq n^2 + 5n^2 = 6n^2$.

(3) $n^2 \notin \mathcal{O}(n)$

Sonst müssten Zahlen n_0 und c existieren mit $n^2 \leq c \cdot n$ für alle $n \geq n_0$.

$\Leftrightarrow n \leq c$

Widerspruch!

O-Notation: Rechenregeln

Im Folgenden schauen wir uns ein paar Rechenregeln an, damit wir eine Funktion $f(n)$ einfacher in eine geeignete Klasse $\mathcal{O}(g(n))$ einordnen können.

Satz (konstante Faktoren):

Für alle Funktionen $f(n)$ und Konstanten $a \in \mathbb{R}$ gilt $a \cdot f(n) \in \mathcal{O}(f(n))$.

Für jede Konstante a ist insbesondere $a \in \mathcal{O}(1)$.

Beweis: Mit $n_0 = 1$ und $c = a$ gilt für alle $n \geq n_0$: $a \cdot f(n) \leq c \cdot f(n)$.

Beispiel: Es ist $3n^2 \in \mathcal{O}(n^2)$.

O-Notation: Rechenregeln

Potenzfunktionen der Form $f(n) = n^k$ mit $k \in \mathbb{N}_0$ wachsen langsamer als Potenzfunktionen mit einem grösseren Exponenten:

Satz (Potenzfunktionen):

Es ist $n^k \in \mathcal{O}(n^l)$ für alle $l, k \in \mathbb{N}_0$ mit $k \leq l$.

Es gilt zudem $n^k \notin \mathcal{O}(n^l)$ falls $l < k$.

Beweis: Bekanntlich gilt $n^k \leq n^l$ für alle natürlichen Zahlen n falls $k \leq l$.

Und $n^k \in \mathcal{O}(n^l)$ für $l < k$ würde bedeuten, dass es Zahlen n_0 und c geben würde mit

$n^k \leq c \cdot n^l$ für alle $n \geq n_0$.

$\Leftrightarrow n^{k-l} \leq c$

Widerspruch!
(da $k - l > 0$)

O-Notation: Rechenregeln

Satz (Summen von Funktionen):

Für zwei Funktionen $f(n), g(n) \in \mathcal{O}(h(n))$ gilt $f(n) + g(n) \in \mathcal{O}(h(n))$.

Beweis: $f(n) \in \mathcal{O}(h(n)) \Rightarrow \exists n_1, c_1 : \forall n \geq n_1 : f(n) \leq c_1 \cdot h(n)$

$g(n) \in \mathcal{O}(h(n)) \Rightarrow \exists n_2, c_2 : \forall n \geq n_2 : g(n) \leq c_2 \cdot h(n)$

$\Rightarrow \forall n \geq \max(n_1, n_2) : f(n) + g(n) \leq c_1 \cdot h(n) + c_2 \cdot h(n) = (c_1 + c_2)h(n)$

$\Rightarrow \exists n_0, c : \forall n \geq n_0 : f(n) + g(n) \leq c \cdot h(n)$

nämlich $n_0 = \max(n_1, n_2)$

nämlich $c = c_1 + c_2$

Aus den bisherigen Rechenregeln folgt, dass bei Polynomen nur die höchste Potenz entscheidend ist:

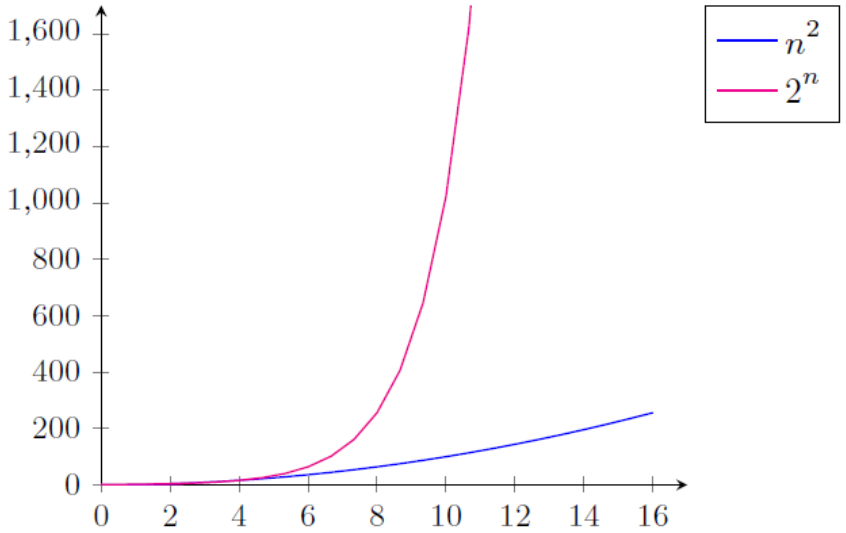
Satz: Es ist $a_k n^k + a_{k-1} n^{k-1} + a_{k-2} n^{k-2} + \ldots + a_1 n + a_0 \in \mathcal{O}(n^k)$

Beispiel: $3n^2 + 4n + 5 \in \mathcal{O}(n^2)$.

O-Notation: Rechenregeln

Exponentialfunktionen wachsen schneller als Potenzfunktionen:

Satz: Für alle $a > 1$ und $b > 0$ gilt $n^b \in \mathcal{O}(a^n)$ und $a^n \notin \mathcal{O}(n^b)$.



Wenn $2^n = m$, dann ist definitionsgemäss $\log_2(m) = n$.

Es ist $2^{10} = 1024$, also $\log_2(1024) = 10$.

Es ist $2^{100} = 1267650600228229401496703205376$, also $\log_2(1267650600228229401496703205376) = 100$.

Als Umkehrfunktion der Exponentialfunktion wächst der Logarithmus dementsprechend extrem(!) langsam.

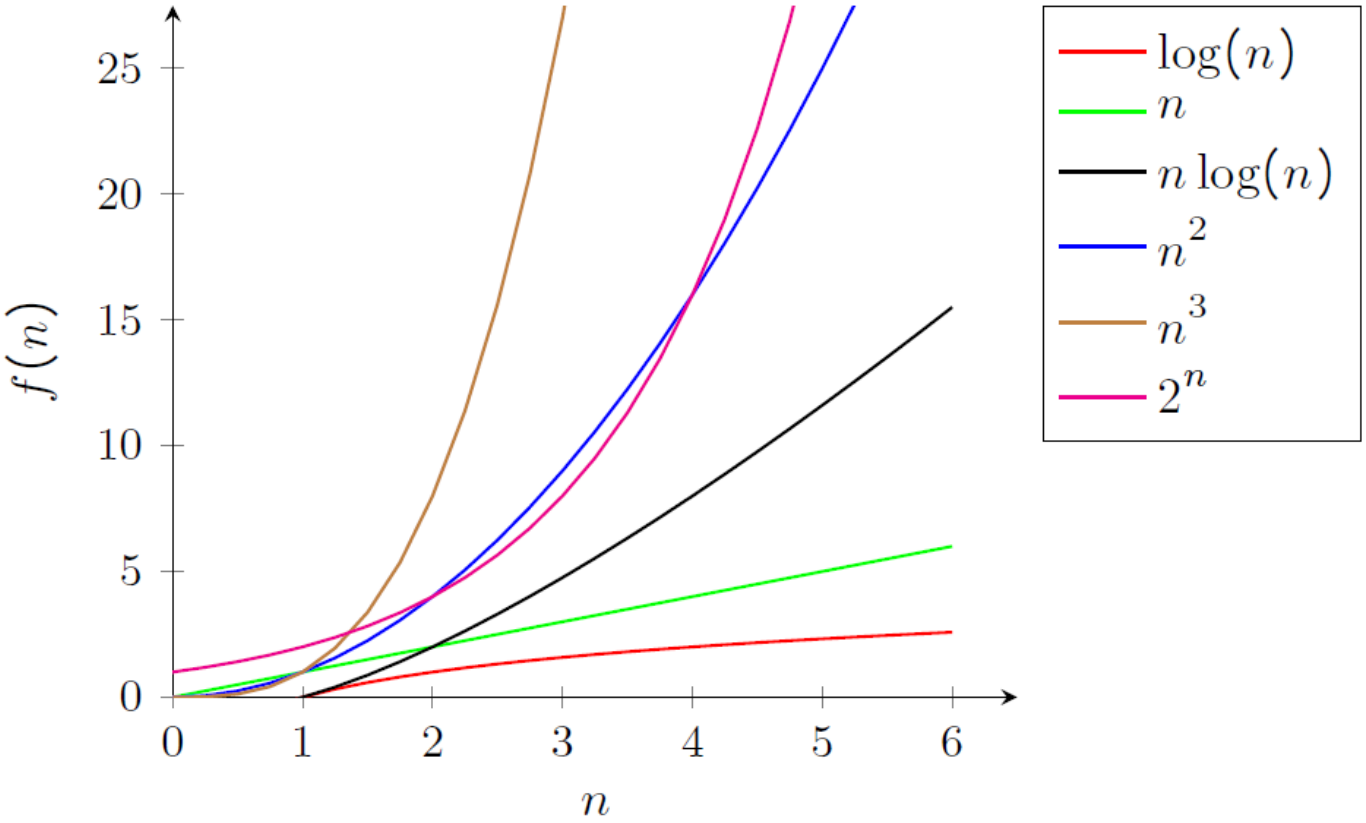
O-Notation: Rechenregeln

Zusammenfassung der wichtigsten Rechenregeln:

Name	**Regel**
Konstante Vorfaktoren	$\forall a \in \mathbb{R}: a \cdot f(n) \in \mathcal{O}(f(n))$
Konstanten	$\forall a \in \mathbb{R}: a \in \mathcal{O}(1)$
Potenzfunktionen	$\forall k, l \in \mathbb{N}_0: \left(k \leq l \implies n^k \in \mathcal{O}\left(n^l\right)\right)$
Summen	$f(n), g(n) \in \mathcal{O}(h(n)) \implies f(n) + g(n) \in \mathcal{O}(h(n))$
Polynome	$a_k n^k + a_{k-1} n^{k-1} + \cdots + a_1 n + a_0 \in \mathcal{O}\left(n^k\right)$
Potenz- und Exponentialfkt.	$\forall a > 1, b > 0: n^b \in O\left(a^n\right)$ $\forall a > 1, b > 0: a^n \notin O\left(n^b\right)$

Zeitkomplexitäten im Vergleich

Verschiedene Zeitkomplexitäten für den Bereich $0 \leq n \leq 6$ im Vergleich:



Anmerkung: Mit $\log(n)$ kürzt man in der Informatik den gebräuchlichsten Logarithmus, den Zweierlogarithmus $\log_2(n)$, ab.

Zeitkomplexitäten im Vergleich

Verschiedene Zeitkomplexitäten für den etwas grösseren Bereich $0 \leq n \leq 16$ im Vergleich:

